Module 6: Data visualization II

PUBHLT 0411 · Python for Public Health Data Analysis

Soumik Purkayastha

Dept. of Biostatistics and Health Data Science, University of Pittsburgh

November 17, 2026

Housekeeping/setup

```
import micropip                                  # installer for packages the browser lacks
await micropip.install("seaborn")                # fetch seaborn; first run takes a few seconds

import numpy as np                               # arrays and arithmetic (Module 3)
import pandas as pd                              # tables (Module 4)
import matplotlib.pyplot as plt                  # figures, axes, labels (Module 5)
import seaborn as sns                            # statistical figures (Module 5)

births = pd.read_csv("data/pa_maternal_infant.csv")        # 67 counties, County Health Rankings 2024
counties = pd.read_csv("data/pa_counties.csv")             # region, metro status, population
joined = births.merge(counties, on="county")               # attach county attributes by key

print(joined.shape)                              # rows, columns: expect (67, 14)
```

```
(67, 14)
```

Every code cell in this deck loads its own data, so any slide runs on its own. In JupyterLab, `import seaborn as sns` is sufficient; the two `micropip` lines are for the browser only.

The maternal and infant data

```python
import pandas as pd                                          # tables

births = pd.read_csv("data/pa_maternal_infant.csv")          # one row per county
counties = pd.read_csv("data/pa_counties.csv")               # metro status per county
joined = births.merge(counties, on="county")                 # one table

cols = ["low_birthweight_pct", "children_in_poverty_pct"]    # the two measures used today
print(joined.groupby("metro")[cols].median())                # median of each, by metro status
```

```
          low_birthweight_pct  children_in_poverty_pct
metro
Metro                     8.1                    14.30
Nonmetro                  7.3                    17.75
```

Source: County Health Rankings & Roadmaps 2024. **Low birthweight** is the percentage of live births under 2,500 grams, 5.3% (Union) to 11.0% (Philadelphia).

Metro counties have the higher median low-birthweight rate, 8.1% against 7.3%, even though child poverty is higher in nonmetro counties.

The question this module opens

Does child poverty account for the metro difference, or does the difference persist at the same level of poverty?

More geoms

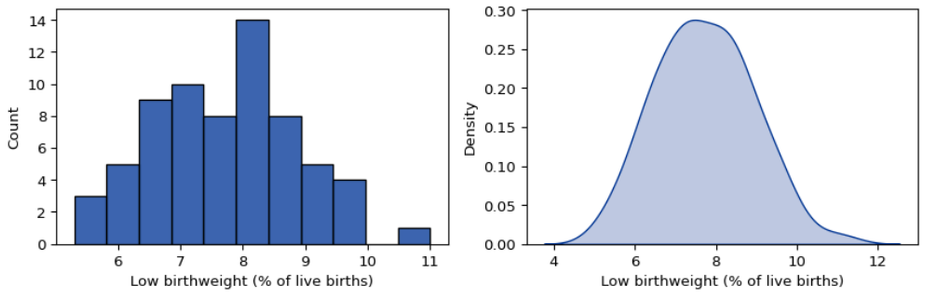
`sns.histplot()` and `sns.kdeplot()`

A histogram counts rows in bins. A **density** curve is a smoothed outline of the same distribution, scaled so its area is 1.

```
import micropip                                                 # installer for browser packages
await micropip.install("seaborn")                               # fetch seaborn
import pandas as pd                                             # tables
import seaborn as sns                                           # statistical figures
import matplotlib.pyplot as plt                                 # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")             # one row per county

fig, (hist, dens) = plt.subplots(1, 2, figsize=(9, 3))          # two panels
sns.histplot(data=births, x="low_birthweight_pct",              # data; low birthweight -> x
             binwidth=0.5,                                      # SETTING: every bin 0.5 points wide
             color="#003594", ax=hist)                          # one colour; left panel
sns.kdeplot(data=births, x="low_birthweight_pct",               # same column, smoothed
            fill=True, color="#003594", ax=dens)                # shaded under the curve; right panel
for ax in (hist, dens):                                         # both panels:
    ax.set_xlabel("Low birthweight (% of live births)")         # x-axis label with units
plt.tight_layout()                                              # stop panels overlapping
plt.show()                                                      # render the figure
```



`binwidth=` and `bins=` are settings: fixed numbers, not columns, so they never go to a mapping.

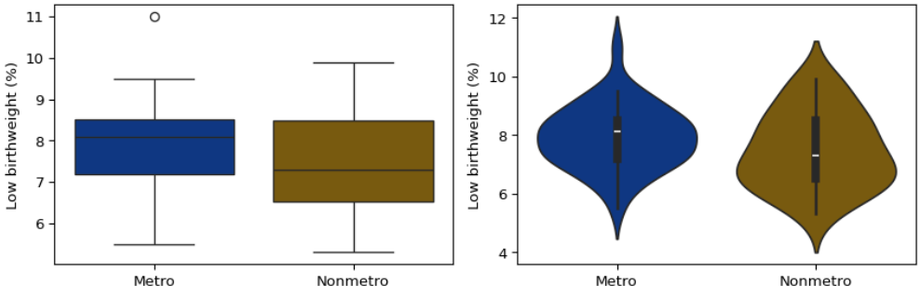
`sns.boxplot()` and `sns.violinplot()`

A violin draws the density of each group on both sides of a centre line, where a boxplot draws only its quartiles.

```
import micropip                                              # installer for browser packages
await micropip.install("seaborn")                            # fetch seaborn
import pandas as pd                                          # tables
import seaborn as sns                                        # statistical figures
import matplotlib.pyplot as plt                              # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")          # one row per county
counties = pd.read_csv("data/pa_counties.csv")               # metro status per county
joined = births.merge(counties, on="county")                 # one table
PAL = {"Metro": "#003594", "Nonmetro": "#8a6400"}            # course colours, by group

fig, (box, vio) = plt.subplots(1, 2, figsize=(9, 3))         # two panels
sns.boxplot(data=joined, x="metro", y="low_birthweight_pct",     # metro -> x, outcome -> y
            hue="metro", palette=PAL, legend=False, ax=box)      # colour by group, no legend
sns.violinplot(data=joined, x="metro", y="low_birthweight_pct", # same mappings
               hue="metro", palette=PAL, legend=False, ax=vio)  # colour by group, no legend
for ax in (box, vio):                                        # both panels:
    ax.set(xlabel="", ylabel="Low birthweight (%)")          # labels
plt.tight_layout()                                           # stop panels overlapping
plt.show()                                                   # render the figure
```



`hue=` repeats the `x` column so each box takes its colour from `palette`; `legend=False` drops the legend, which would only repeat the axis labels.

seaborn's three families

Every seaborn mark belongs to a family, and each family has a **figure-level** function that can split the data into panels.

Family	Figure-level	Axes-level marks
Relational: two numeric variables	`sns.relplot()`	`scatterplot`, `lineplot`
Distributional: one numeric variable	`sns.displot()`	`histplot`, `kdeplot`
Categorical: numeric by category	`sns.catplot()`	`boxplot`, `violinplot`, `countplot`, `barplot`

Axes-level functions draw on one `ax=`. **Figure-level** functions create their own figure and take `col=` and `row=` instead. The seaborn gallery shows a worked example of each.

Layers and statistical transformations

Layering: order matters

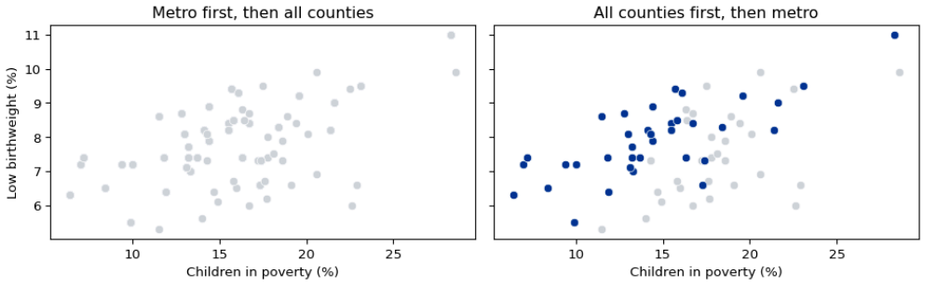
Each call draws on top of the previous one, so the layer that must be seen goes **last**.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table
metro = joined[joined["metro"] == "Metro"]               # the group to highlight
xy = dict(x="children_in_poverty_pct", y="low_birthweight_pct", s=40)  # shared mappings, size

fig, (bad, good) = plt.subplots(1, 2, figsize=(10, 3.2), sharey=True)  # two panels
sns.scatterplot(data=metro, **xy, color="#003594", ax=bad)   # layer 1: metro counties
sns.scatterplot(data=joined, **xy, color="#d1d5db", ax=bad)  # layer 2: all 67, in grey, on top
bad.set_title("Metro first, then all counties")          # panel title

sns.scatterplot(data=joined, **xy, color="#d1d5db", ax=good) # layer 1: all 67 as a backdrop
sns.scatterplot(data=metro, **xy, color="#003594", ax=good)  # layer 2: metro counties on top
good.set_title("All counties first, then metro")         # panel title
for ax in (bad, good):                                   # both panels:
    ax.set(xlabel="Children in poverty (%)", ylabel="Low birthweight (%)")  # labels with units
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



`zorder=` overrides the order: a layer with a higher `zorder` is drawn in front.

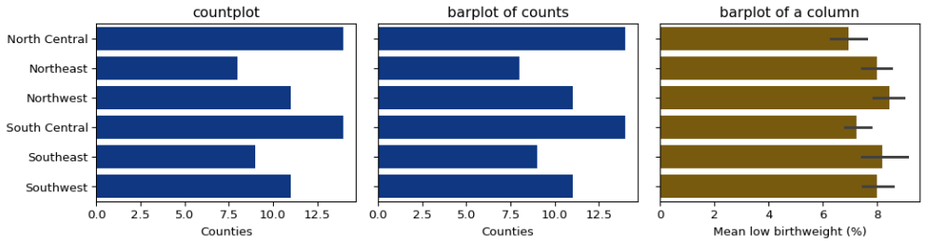
Statistical transformations

Many marks compute something before drawing. `countplot` counts rows; `barplot` computes a **mean** of `y` and adds an error bar by default.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # region per county
joined = births.merge(counties, on="county")             # one table
n = joined["region"].value_counts().reset_index()        # precomputed counts per region
order = sorted(joined["region"].unique())                # one region order for all panels

fig, (cnt, pre, avg) = plt.subplots(1, 3, figsize=(11, 3), sharey=True)  # three panels
sns.countplot(data=joined, y="region", order=order,      # seaborn counts the rows itself
              color="#003594", ax=cnt)                   # one colour
cnt.set(title="countplot", xlabel="Counties")            # panel title; x label
sns.barplot(data=n, y="region", x="count", order=order,  # bar length = a column of counts
            color="#003594", ax=pre)                     # one colour
pre.set(title="barplot of counts", xlabel="Counties")    # panel title; x label
sns.barplot(data=joined, y="region", x="low_birthweight_pct",  # bar length = a MEAN per region
            order=order, color="#8a6400", ax=avg)        # same order; one colour
avg.set(title="barplot of a column",                     # panel title
        xlabel="Mean low birthweight (%)")               # what the bar now shows
for ax in (cnt, pre, avg):                               # every panel:
    ax.set_ylabel("")                                    # region names need no label
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



`errorbar=None` removes the error bar. The interval it draws is inference, which this course does not cover.

One summary, or one per group

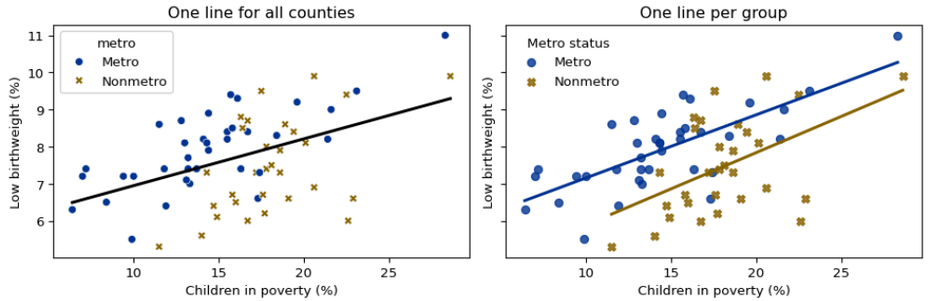
A mapping applies to every layer that receives it. A layer without `hue=` summarises all rows at once.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table
PAL = {"Metro": "#003594", "Nonmetro": "#8a6400"}        # course colours, by group
xy = dict(x="children_in_poverty_pct", y="low_birthweight_pct")  # shared mappings

fig, (one, each) = plt.subplots(1, 2, figsize=(10, 3.4), sharey=True)  # two panels, one y scale
sns.scatterplot(data=joined, **xy, hue="metro", style="metro",          # points by group
                palette=PAL, ax=one)                                    # left panel
sns.regplot(data=joined, **xy, scatter=False, ci=None,     # ONE straight line through all rows
            color="black", ax=one)                         # no hue, so no grouping
one.set_title("One line for all counties")                 # panel title

for group, colour in PAL.items():                          # one layer per group:
    rows = joined[joined["metro"] == group]                # that group's counties
    sns.regplot(data=rows, **xy, ci=None, color=colour,    # points and a line per group
                marker="o" if group == "Metro" else "X",   # second channel: marker shape
                label=group, ax=each)                      # legend entry
each.set_title("One line per group")                       # panel title
each.legend(title="Metro status", frameon=False)           # legend
for ax in (one, each):                                     # both panels:
    ax.set(xlabel="Children in poverty (%)", ylabel="Low birthweight (%)")  # labels with units
plt.tight_layout()                                         # stop panels overlapping
plt.show()                                                 # render the figure
```



Facets

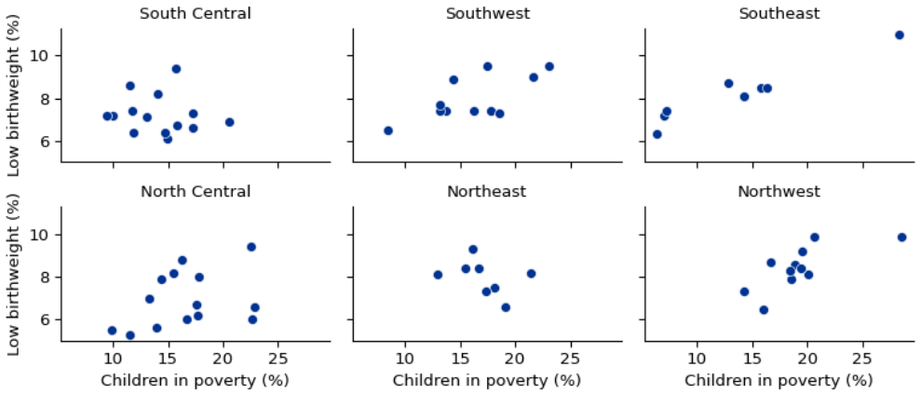
Facets: one panel per group

`col=` gives each value of a column its own panel. `col_wrap=` wraps a long row of panels onto several lines.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # region per county
joined = births.merge(counties, on="county")             # one table

g = sns.relplot(data=joined,                             # figure-level: builds its own figure
                x="children_in_poverty_pct",             # poverty -> x
                y="low_birthweight_pct",                 # low birthweight -> y
                col="region", col_wrap=3,                # one panel per region, 3 per row
                height=1.8, aspect=1.5,                  # each panel 1.8 in tall, 1.5x as wide
                color="#003594")                         # one colour for every point
g.set_titles("{col_name}")                               # panel title = the region alone
g.set_axis_labels("Children in poverty (%)",             # x-axis label, once per column
                  "Low birthweight (%)")                 # y-axis label, once per row
plt.show()                                               # render the figure
```



`relplot` returns a **FacetGrid**, `g`, whose methods label every panel at once.

Facet grids: two conditioning variables

`row=` and `col=` together give one panel per **combination**. A numeric column becomes a condition once it is cut into groups.

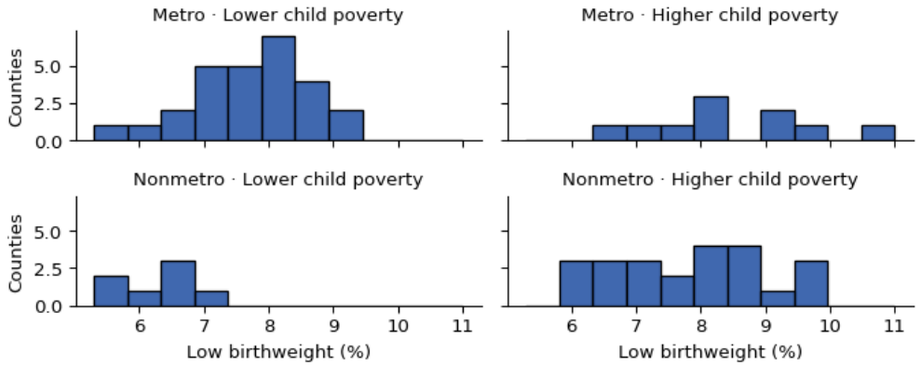
```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import numpy as np                                       # np.where
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table

cut = joined["children_in_poverty_pct"].median()         # statewide median: 16.1%
joined["poverty"] = np.where(                            # a new categorical column:
    joined["children_in_poverty_pct"] > cut,             # above the median ...
    "Higher child poverty", "Lower child poverty")       # ... or not

g = sns.displot(data=joined, x="low_birthweight_pct",    # distribution of low birthweight
                row="metro", col="poverty",              # metro by row, poverty by column
                binwidth=0.5, height=1.5, aspect=2.4,    # bin width; panel size
                color="#003594")                         # one colour
g.set_titles("{row_name} · {col_name}")                  # both conditions in each title
g.set_axis_labels("Low birthweight (%)", "Counties")     # axis labels
plt.show()                                               # render the figure

print(joined.groupby(["metro", "poverty"])["low_birthweight_pct"].median())  # median per panel
```




```
metro     poverty
Metro     Higher child poverty    8.35
          Lower child poverty     7.70
Nonmetro  Higher child poverty    7.90
          Lower child poverty     6.40
Name: low_birthweight_pct, dtype: float64
```

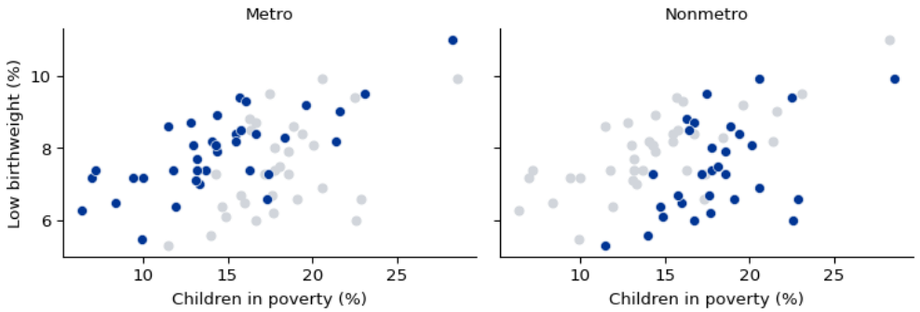
Facets versus aesthetics

A facet removes the other groups from view. Drawing all rows in grey behind each panel restores the comparison.

```
import micropip                                              # installer for browser packages
await micropip.install("seaborn")                            # fetch seaborn
import pandas as pd                                          # tables
import seaborn as sns                                        # statistical figures
import matplotlib.pyplot as plt                              # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")          # one row per county
counties = pd.read_csv("data/pa_counties.csv")               # metro status per county
joined = births.merge(counties, on="county")                 # one table
x, y = "children_in_poverty_pct", "low_birthweight_pct"      # the two columns

g = sns.relplot(data=joined, x=x, y=y, col="metro",          # one panel per metro group
                height=2.8, aspect=1.4, color="#003594",     # panel size; group colour
                zorder=2)                                    # draw these points in front
for ax in g.axes.flat:                                       # every panel:
    ax.scatter(joined[x], joined[y], color="#d1d5db",        # all 67 counties in light grey
               s=25, zorder=1)                               # ... behind the group's points
g.set_titles("{col_name}")                                   # panel title = the group
g.set_axis_labels("Children in poverty (%)", "Low birthweight (%)")  # axis labels
plt.show()                                                   # render the figure
```



`g.axes.flat` reaches each panel's matplotlib axes, so any matplotlib call can add to it.

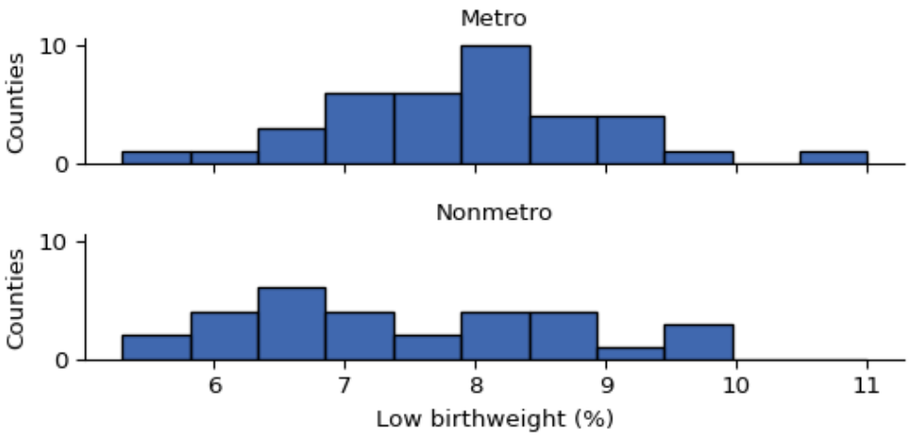
Rows or columns; shared axes

Panels **stacked in a column** share an x-axis, so their distributions line up. Panels **side by side** share a y-axis.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table

g = sns.displot(data=joined, x="low_birthweight_pct",    # distribution of low birthweight
                row="metro",                             # one ROW per group: x-axes align
                binwidth=0.5, height=1.4, aspect=4,      # bin width; wide, short panels
                color="#003594")                         # one colour
g.set_titles("{row_name}")                               # panel title = the group
g.set_axis_labels("Low birthweight (%)", "Counties")     # axis labels
plt.show()                                               # render the figure
```



`sharex=False` or `sharey=False` gives each panel its own scale, at the cost of the comparison.

Position adjustments

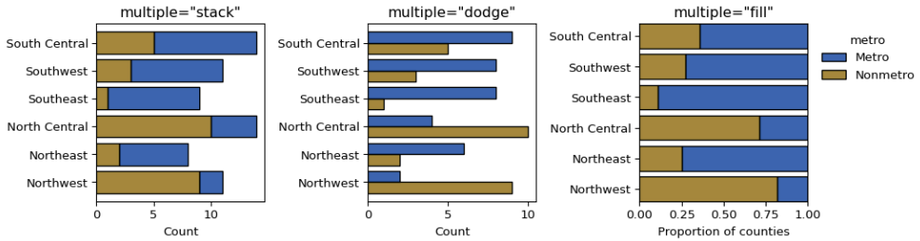
Position adjustments: bars

When groups overlap in one bar, `multiple=` decides where each goes: **stack**, **dodge** (side by side), or **fill** (stack, scaled to proportions).

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

counties = pd.read_csv("data/pa_counties.csv")           # region and metro per county
PAL = {"Metro": "#003594", "Nonmetro": "#8a6400"}        # course colours, by group

fig, axes = plt.subplots(1, 3, figsize=(11, 3), sharey=False)  # three panels
for ax, how in zip(axes, ["stack", "dodge", "fill"]):    # one adjustment per panel
    sns.histplot(data=counties, y="region", hue="metro",   # counts of counties per region
                 multiple=how, shrink=0.8,                 # the adjustment; gaps between bars
                 palette=PAL, legend=(how == "fill"),      # course colours; one legend only
                 ax=ax)                                    # this panel
    ax.set(title=f'multiple="{how}"', ylabel="")           # panel title; names on the left
axes[2].set_xlabel("Proportion of counties")             # fill: the scale is a proportion
sns.move_legend(axes[2], "upper left",                   # legend outside the last panel,
                bbox_to_anchor=(1.02, 1), frameon=False)   # so it covers no bars
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



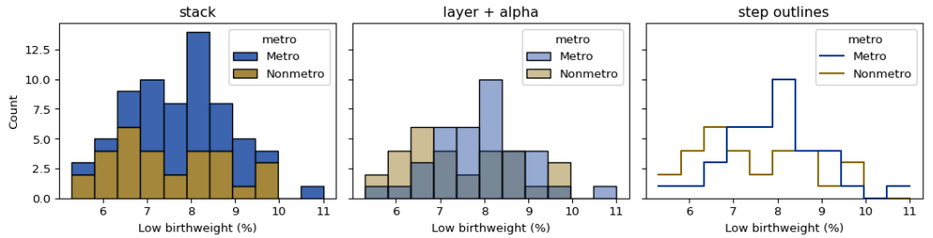
Position adjustments: histograms

Stacked histograms hide the shape of every group except the bottom one. **Layering** with transparency, or drawing outlines with `element="step"`, keeps each group fully visible.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table
PAL = {"Metro": "#003594", "Nonmetro": "#8a6400"}        # course colours, by group
opts = dict(data=joined, x="low_birthweight_pct", hue="metro",  # shared data and mappings
            binwidth=0.5, palette=PAL)                   # bin width; course colours

fig, axes = plt.subplots(1, 3, figsize=(11, 3), sharey=True)    # three panels, one y scale
sns.histplot(**opts, multiple="stack", ax=axes[0])       # default for hue: stacked
sns.histplot(**opts, multiple="layer", alpha=0.4, ax=axes[1])   # overlaid, see-through
sns.histplot(**opts, element="step", fill=False, ax=axes[2])    # outlines only
for ax, t in zip(axes, ["stack", "layer + alpha", "step outlines"]):  # each panel:
    ax.set(title=t, xlabel="Low birthweight (%)")        # title and x-axis label
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



Scales

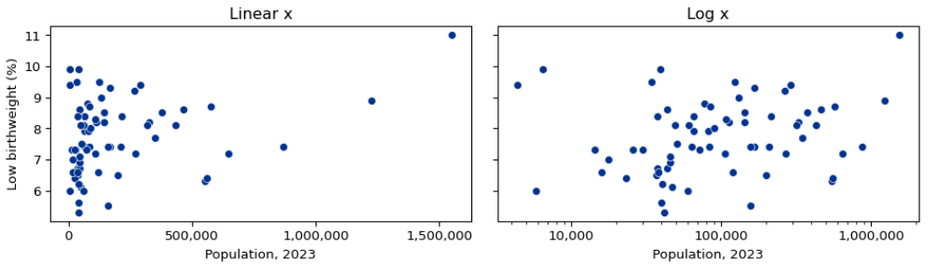
Scales: ticks, labels, and log axes

A scale maps data values to positions. matplotlib sets ticks automatically; `set_xscale("log")` spaces values by ratio, and a **locator** fixes the tick spacing.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes
from matplotlib.ticker import MultipleLocator, StrMethodFormatter  # tick spacing and format

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # population per county
joined = births.merge(counties, on="county")             # one table

fig, (lin, log) = plt.subplots(1, 2, figsize=(10, 3), sharey=True)  # two panels
log.set_xscale("log")                                    # equal ratios -> equal distances
for ax in (lin, log):                                    # same scatter in both panels
    sns.scatterplot(data=joined, x="population_2023",    # population -> x
                    y="low_birthweight_pct",             # low birthweight -> y
                    color="#003594", ax=ax)              # one colour
    ax.xaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))  # 1500000 -> 1,500,000
    ax.set_xlabel("Population, 2023")                    # x-axis label
lin.xaxis.set_major_locator(MultipleLocator(500000))     # an x tick every 500,000 people
lin.yaxis.set_major_locator(MultipleLocator(1))          # a y tick every 1 point
lin.set_ylabel("Low birthweight (%)")                    # y-axis label with units
lin.set_title("Linear x"); log.set_title("Log x")        # panel titles
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



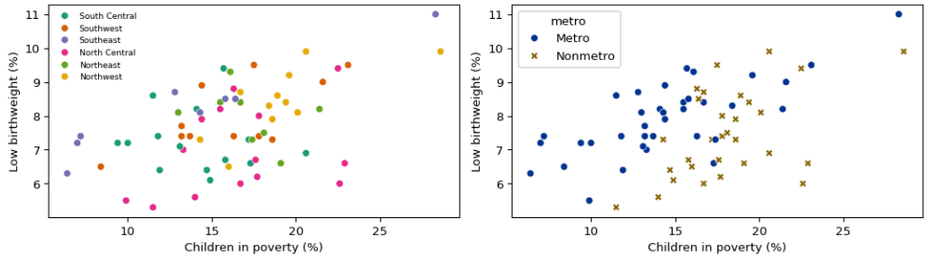
Colour scales: named and manual palettes

`palette=` takes either a **named** palette, such as the ColorBrewer set `"Dark2"`, or a **dictionary** from each group to a colour.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # region and metro per county
joined = births.merge(counties, on="county")             # one table
xy = dict(data=joined, x="children_in_poverty_pct", y="low_birthweight_pct")  # shared mappings

fig, (named, manual) = plt.subplots(1, 2, figsize=(11, 3.2))    # two panels
sns.scatterplot(**xy, hue="region", palette="Dark2", ax=named)  # a named ColorBrewer palette
sns.scatterplot(**xy, hue="metro", style="metro",               # metro -> colour and shape
                palette={"Metro": "#003594",                    # manual: group -> colour
                         "Nonmetro": "#8a6400"}, ax=manual)     # dark gold for nonmetro
named.legend(fontsize=7, frameon=False)                  # compact legend
for ax in (named, manual):                               # both panels:
    ax.set(xlabel="Children in poverty (%)", ylabel="Low birthweight (%)")  # labels with units
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



`sns.color_palette("Dark2")` displays a named palette's colours.

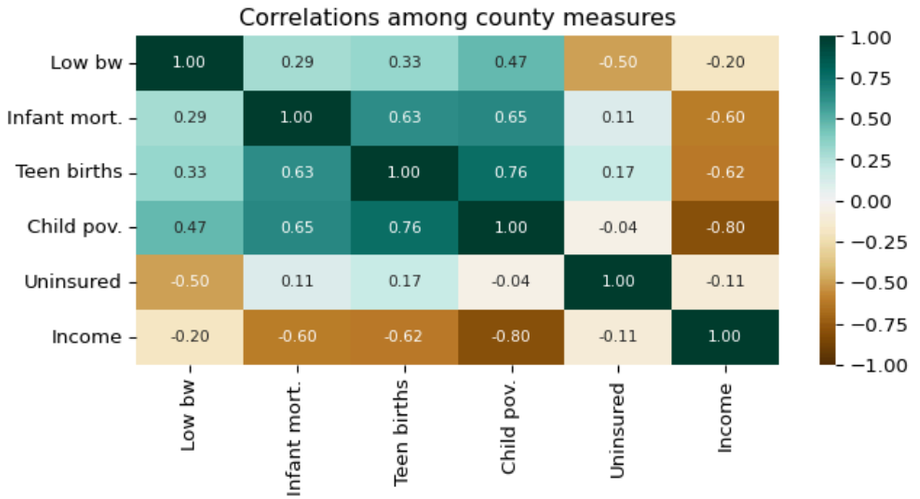
Diverging colour scales: a correlation heatmap

A **diverging** palette has a neutral centre and two directions, which suits a value with a meaningful zero.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
measures = births[["low_birthweight_pct", "infant_mortality_per_1000",  # six measures:
                   "teen_births_per_1000", "children_in_poverty_pct",   # births and poverty
                   "uninsured_children_pct", "median_household_income"]]  # insurance, income
short = ["Low bw", "Infant mort.", "Teen births",        # short labels for the cells,
         "Child pov.", "Uninsured", "Income"]            # in the same order as the columns

fig, ax = plt.subplots(figsize=(7, 3.8))                 # canvas and one axes
sns.heatmap(measures.corr(), annot=True, fmt=".2f",      # correlations, printed in each cell
            cmap="BrBG", vmin=-1, vmax=1,                # diverging palette, fixed at -1..+1
            xticklabels=short, yticklabels=short,        # readable labels
            annot_kws={"size": 8}, ax=ax)                # small cell text; target axes
ax.set_title("Correlations among county measures")        # title
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```



Themes

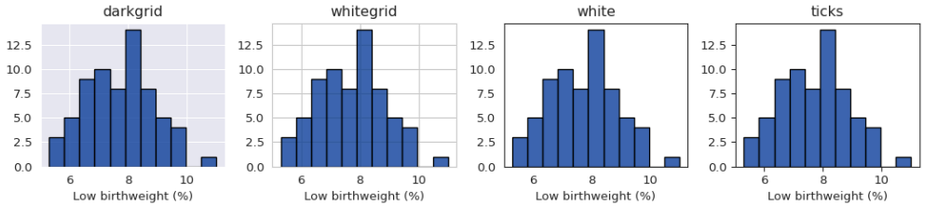
Built-in styles

A theme sets the **non-data** elements: background, grid, and ticks. seaborn has five styles.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county

fig = plt.figure(figsize=(11, 2.6))                      # an empty canvas
for i, style in enumerate(["darkgrid", "whitegrid",      # four of the five styles
                           "white", "ticks"], start=1):  # ("dark" is the fifth)
    with sns.axes_style(style):                          # applies only inside this block
        ax = fig.add_subplot(1, 4, i)                    # axes created in that style
    sns.histplot(data=births, x="low_birthweight_pct",   # the same histogram each time
                 binwidth=0.5, color="#003594", ax=ax)   # bin width; one colour
    ax.set(title=style, xlabel="Low birthweight (%)",    # style name as the title
           ylabel="")                                    # no y label, to save room
plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



`sns.set_theme(style="white")` sets the style for **every later figure** in the session.

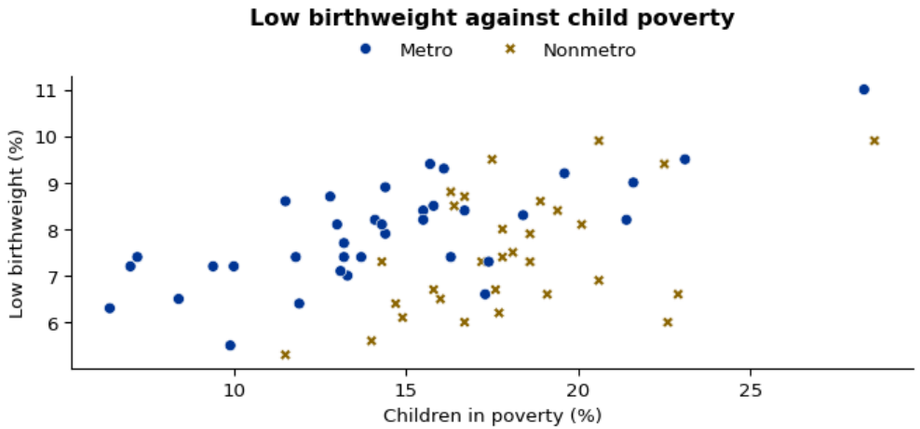
Adjusting single elements

Individual elements change through matplotlib and two seaborn helpers: `sns.despine()` and `sns.move_legend()`.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = births.merge(counties, on="county")             # one table

fig, ax = plt.subplots(figsize=(7, 3.4))                 # canvas and one axes
sns.scatterplot(data=joined, x="children_in_poverty_pct",  # poverty -> x
                y="low_birthweight_pct",                 # low birthweight -> y
                hue="metro", style="metro",              # metro -> colour and shape
                palette={"Metro": "#003594", "Nonmetro": "#8a6400"}, ax=ax)  # course colours
sns.despine(ax=ax)                                       # remove the top and right borders
sns.move_legend(ax, "lower center",                      # legend centred ...
                bbox_to_anchor=(0.5, 1.0), ncol=2,       # ... above the axes, in one row
                title=None, frameon=False)               # no title, no box
ax.set_title("Low birthweight against child poverty",    # title ...
             fontweight="bold", pad=28)                  # ... in bold, clear of the legend
ax.set(xlabel="Children in poverty (%)", ylabel="Low birthweight (%)")  # labels with units
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```



`legend=False` in the plotting call removes the legend entirely.

Combining and saving

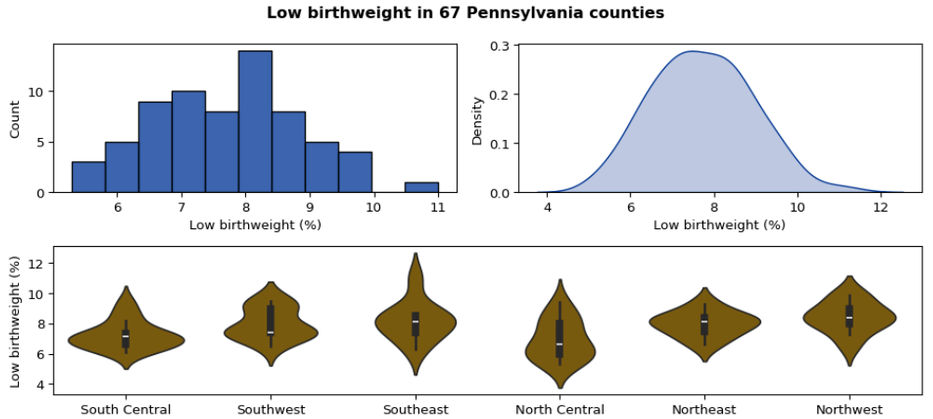
Combining figures

`plt.subplot_mosaic()` lays out named panels from a sketch: each list is a row, and a repeated name spans several cells.

```python
import micropip                                              # installer for browser packages
await micropip.install("seaborn")                            # fetch seaborn
import pandas as pd                                          # tables
import seaborn as sns                                        # statistical figures
import matplotlib.pyplot as plt                              # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")          # one row per county
counties = pd.read_csv("data/pa_counties.csv")               # region per county
joined = births.merge(counties, on="county")                 # one table
LBW = "low_birthweight_pct"                                  # the column every panel uses

fig, axd = plt.subplot_mosaic([["hist", "dens"],             # top row: two panels
                               ["vio", "vio"]],              # bottom row: one wide panel
                              figsize=(10, 4.6))             # canvas size
sns.histplot(data=joined, x=LBW, binwidth=0.5,               # top left: histogram
             color="#003594", ax=axd["hist"])                # one colour
sns.kdeplot(data=joined, x=LBW, fill=True,                   # top right: density
            color="#003594", ax=axd["dens"])                 # one colour
sns.violinplot(data=joined, x="region", y=LBW,               # bottom: one violin per region
               color="#8a6400", ax=axd["vio"])               # one colour
for name in ("hist", "dens"):                                # top panels:
    axd[name].set_xlabel("Low birthweight (%)")              # x-axis label with units
axd["vio"].set(xlabel="", ylabel="Low birthweight (%)")      # bottom panel labels
fig.suptitle("Low birthweight in 67 Pennsylvania counties",  # one title for the figure
             fontweight="bold")                              # in bold
plt.tight_layout()                                           # stop panels overlapping
plt.show()                                                   # render the figure
```



Saving figures

`fig.savefig()` writes the figure to a file. The extension chooses the format.

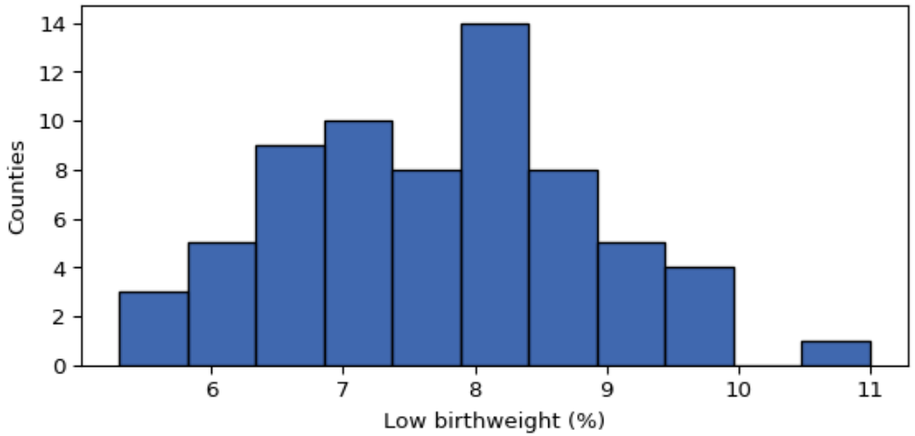
```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import os                                                # file sizes
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

births = pd.read_csv("data/pa_maternal_infant.csv")      # one row per county

fig, ax = plt.subplots(figsize=(6, 3))                   # canvas and one axes
sns.histplot(data=births, x="low_birthweight_pct",       # the figure to save
             binwidth=0.5, color="#003594", ax=ax)       # bin width; one colour
ax.set(xlabel="Low birthweight (%)", ylabel="Counties")  # labels
plt.tight_layout()                                       # keep labels inside the canvas

fig.savefig("lbw_hist.png", dpi=300, bbox_inches="tight")  # raster image, 300 dots per inch
fig.savefig("lbw_hist.pdf", bbox_inches="tight")           # vector image, sharp at any size
plt.show()                                               # render AFTER saving

for f in ["lbw_hist.png", "lbw_hist.pdf"]:               # confirm both files exist
    print(f, os.path.getsize(f), "bytes")                # name and size
```




```
lbw_hist.png 35326 bytes
lbw_hist.pdf 11917 bytes
```

`savefig()` comes **before** `plt.show()`: after showing, the figure may be empty.

Appendix: why visualize?

Identical summaries, different data

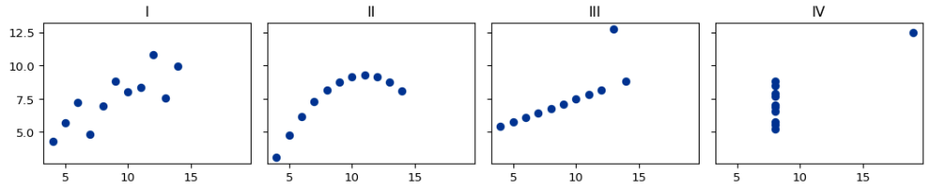
Anscombe's quartet (1973): four data sets with the same means, variances, and correlation.

```python
import pandas as pd                                          # tables
import matplotlib.pyplot as plt                              # figure and axes

x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]                   # x for sets I–III
quartet = {                                                  # set name -> (x, y)
    "I":   (x, [8.04, 6.95, 7.58, 8.81, 8.33, 9.96,         # set I: y values ...
                7.24, 4.26, 10.84, 4.82, 5.68]),             # ... eleven in all
    "II":  (x, [9.14, 8.14, 8.74, 8.77, 9.26, 8.10,         # set II
                6.13, 3.10, 9.13, 7.26, 4.74]),              # ... eleven in all
    "III": (x, [7.46, 6.77, 12.74, 7.11, 7.81, 8.84,        # set III
                6.08, 5.39, 8.15, 6.42, 5.73]),              # ... eleven in all
    "IV":  ([8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8],             # set IV has its own x
            [6.58, 5.76, 7.71, 8.84, 8.47, 7.04,            # set IV: y values ...
             5.25, 12.50, 5.56, 7.91, 6.89]),               # ... eleven in all
}                                                            # end of the four sets

fig, axes = plt.subplots(1, 4, figsize=(11, 2.4), sharex=True, sharey=True)  # four panels
for ax, (name, (xs, ys)) in zip(axes, quartet.items()):   # one set per panel
    ax.scatter(xs, ys, color="#003594")                      # the eleven points
    ax.set_title(name)                                       # panel title
    s = pd.Series(ys)                                        # y as a Series, for summaries
    print(name, "mean y", round(s.mean(), 2),                # same mean ...
          "| sd y", round(s.std(), 2),                       # ... same spread ...
          "| r", round(pd.Series(xs).corr(s), 3))           # ... same correlation
plt.tight_layout()                                           # stop panels overlapping
plt.show()                                                   # render the figure
```

```
I mean y 7.5 | sd y 2.03 | r 0.816
II mean y 7.5 | sd y 2.03 | r 0.816
III mean y 7.5 | sd y 2.03 | r 0.816
IV mean y 7.5 | sd y 2.03 | r 0.817
```



A figure of counts often draws a map of population

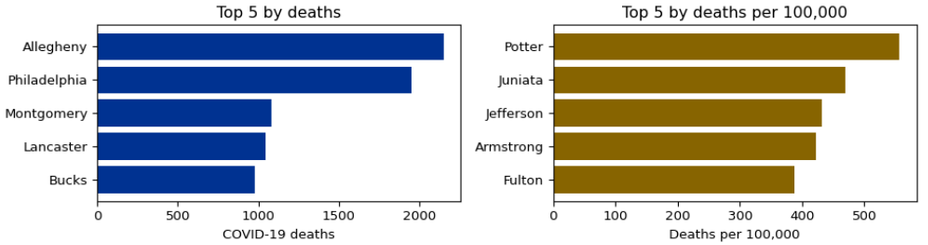
```python
import pandas as pd                                          # tables
import matplotlib.pyplot as plt                              # figure and axes

covid = pd.read_csv("data/pa_covid_monthly.csv")             # deaths by county and month
counties = pd.read_csv("data/pa_counties.csv")               # population per county
covid = covid[covid["month"] >= "2021-08"]                   # keep the reliably reported period

totals = (covid.groupby("county", as_index=False)["covid_deaths"].sum()  # deaths per county
               .merge(counties[["county", "population_2023"]],           # attach population
                      on="county"))                                      # by county
totals["per_100k"] = (totals["covid_deaths"]                 # deaths ...
                      / totals["population_2023"] * 100000)  # ... per 100,000 residents

fig, (cnt, rate) = plt.subplots(1, 2, figsize=(10, 2.8))   # two panels
top = totals.nlargest(5, "covid_deaths").sort_values("covid_deaths")  # top 5 by count
cnt.barh(top["county"], top["covid_deaths"], color="#003594")         # bars by count
cnt.set(title="Top 5 by deaths", xlabel="COVID-19 deaths")            # labels
top = totals.nlargest(5, "per_100k").sort_values("per_100k")          # top 5 by rate
rate.barh(top["county"], top["per_100k"], color="#8a6400")            # bars by rate
rate.set(title="Top 5 by deaths per 100,000", xlabel="Deaths per 100,000")  # labels
plt.tight_layout()                                           # stop panels overlapping
plt.show()                                                   # render the figure

pop = totals["population_2023"]                              # the denominator
print("count vs population:", round(totals["covid_deaths"].corr(pop), 3))  # counts track size
print("rate  vs population:", round(totals["per_100k"].corr(pop), 3))      # rates do not
```




```
count vs population: 0.953
rate  vs population: -0.308
```

The two lists share no counties. Deaths track population at r = +0.953; the rate does not.

Session outcomes

1. More marks: `histplot`, `kdeplot`, `boxplot`, `violinplot`, and seaborn's three families
2. **Layering** in order, and `zorder` when the order must change
3. **Statistical transformations**: what `countplot`, `barplot`, and `regplot` compute
4. **Facets** with `relplot` and `displot`: `col=`, `row=`, `col_wrap=`, shared axes
5. **Position adjustments**: `multiple="stack" / "dodge" / "fill" / "layer"`
6. **Scales**: log axes, tick locators, named and manual palettes, diverging heatmaps
7. **Themes**: `axes_style`, `despine`, `move_legend`; combining with `subplot_mosaic`; `savefig`
8. Figures before summaries; **rates before counts**

> **Lab 6, now**
>
> Questions 1–3 in class (debug, facets and position, themes). All eight questions are due **Thursday 11/19 at 11:59 pm**.

That’s all folks!

On to Lab 6.